SRE HANDBOOK

VERIQTA

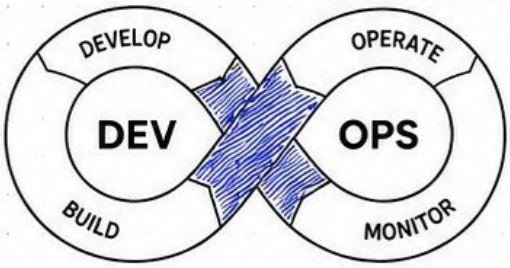
ENGINEERING EXCELLENCE

RELIABILITY AT SCALE BY DESIGN

SITE RELIABILITY ENGINEERING HANDBOOK

- SLOs & ERROR BUDGETS
- OBSERVABILITY
- INCIDENTS
- AUTOMATION
- RESILIENCE
- PRODUCTION EXCELLENCE

FROM SRE FOUNDATIONS TO **PRODUCTION RELIABILITY**




MEASURE WHAT MATTERS


SET SLOs ACHIEVE RELIABILITY


DETECT EARLY


RESPOND EFFECTIVELY


LEARN AND IMPROVE

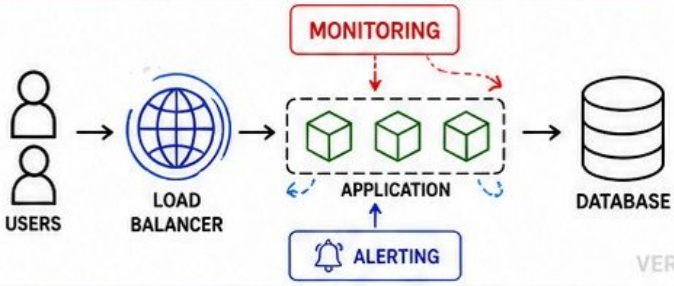

BUILD RESILIENT SYSTEMS

WHAT YOU WILL LEARN

- SRE PRINCIPLES AND FUNDAMENTALS
- SLIs, SLOs, AND ERROR BUDGETS
- OBSERVABILITY AND MONITORING
- ALERTING AND INCIDENT MANAGEMENT
- POSTMORTEMS AND CONTINUOUS LEARNING
- AUTOMATION AND TOIL REDUCTION
- ON-CALL AND INCIDENT READINESS
- RESILIENCE AND FAILURE ENGINEERING
- CAPACITY PLANNING AND SCALING
- KUBERNETES AND CLOUD RELIABILITY


BUILD RELIABLE SYSTEMS. DELIVER VALUE. EARN TRUST. **AT SCALE.**

- REAL PRODUCTION LABS
- BEST PRACTICES
- CHECKLISTS
- CHEAT SHEETS
- SENIOR ENGINEER INSIGHTS



RELIABILITY IS AN ENGINEERING DISCIPLINE

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

• VERIQTA •

PAGE 1

VERIQTA
ENGINEERING EXCELLENCE

SRE HANDBOOK

1. INTRODUCTION TO SITE RELIABILITY ENGINEERING

Building reliable systems. Delivering trust at scale.

RELIABILITY IS NOT A FEATURE, IT IS A FOUNDATION.

WHAT IS SRE?

Site Reliability Engineering (SRE) is a discipline that applies software engineering practices to infrastructure and operations problems to build and run highly reliable systems at scale.

SRE PRINCIPLE

Measure reliability. Set targets.
Automate relentlessly. Learn from failure.
Improve continuously.

ORIGINS AND PURPOSE OF SRE

- Created by Google in 2003 to solve reliability problems at massive scale.
- Recognized that operations problems could be solved with engineering discipline.
- Purpose: To reliably deliver services users love while enabling rapid innovation.
- Goal: Maximize reliability while minimizing toil.

SRE vs DEVOPS

DEVOPS	SRE
Culture and collaboration	Discipline and role
Bridges development and operations	Focus on reliability and scalability
Focus on delivery speed	Engineering approach to operations
Shared responsibility	Defined practices and metrics

SRE vs TRADITIONAL OPERATIONS

TRADITIONAL OPS	SRE
Manual processes	Automated processes
Reactive to problems	Proactive and data-driven
Focus on uptime only	Focus on reliability and user experience
Siloed teams	Collaborative teams
Tools over automation	Automation over tools
Change-averse culture	Learning culture

RELIABILITY AS AN ENGINEERING PROBLEM

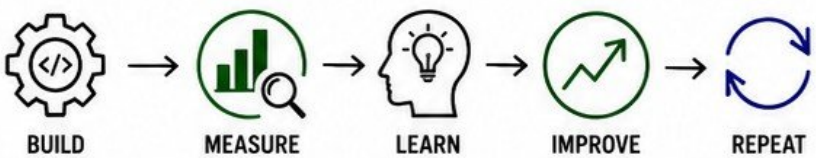
- Reliability is measurable and testable.
- Systems will fail—design for failure.
- Use data to understand behavior.
- Make trade-offs based on risk and impact.
- Engineer systems that degrade gracefully.

CORE SRE RESPONSIBILITIES

1. Define and track SLIs, SLOs, and error budgets
2. Build and maintain observability (metrics, logs, traces)
3. Design for reliability, scalability, and performance
4. Automate to reduce toil and human error
5. Respond to incidents and minimize impact
6. Conduct blameless postmortems and drive improvements
7. Capacity planning and cost optimization

PRODUCTION RELIABILITY MINDSET

- Users come first.
- Reliability is a feature.
- Embrace risk, control risk.
- Data over opinions.
- Learn from every failure.
- Strive for continuous improvement.



SRE IS A JOURNEY OF CONTINUOUS RELIABILITY.
THINK LONG TERM. MAKE IMPACT. DELIVER TRUST.

VERIQTA
ENGINEERING EXCELLENCE

2. RELIABILITY ENGINEERING FUNDAMENTALS

Building reliable systems that users can trust.

RELIABILITY IS EARNED. TRUST IS BUILT.

1 RELIABILITY

The probability that a system performs its intended function without failure for a given period of time in a given environment.

Key Points

- Consistency over time
- Measurable and testable
- Based on real user experience
- Depends on many factors

2 AVAILABILITY

The proportion of time a system is operational and accessible when required for use.

Key Points

- Usually expressed as a %
- Example: 99.9% availability = ~8.76 hours downtime/year
- Critical for user trust

3 DURABILITY

The ability of a system to retain data accurately and consistently over time.

Key Points

- Data is not lost or corrupted
- Survives crashes and disasters
- Backups and replication improve durability

4 RESILIENCE

The ability of a system to recover quickly from failures, adapting to changes while continuing to function.

Key Points

- Absorb shocks and adapt
- Recover and continue
- Key to long-term stability

5 FAULT TOLERANCE

The ability of a system to continue operating properly in the event of failures of its components.

Key Points

- Detect failures
- Isolate faulty components
- Continue service
- No single point of failure

6 HIGH AVAILABILITY

System architecture and practices designed to keep services available with minimal downtime.

Key Points

- Redundant components
- Failover and redundancy
- Load balancing
- Minimize downtime

7 RELIABILITY VS PERFORMANCE

Performance is how fast the system works. Reliability is how consistently it works.

Key Points

- High performance can reduce reliability if not balanced
- Optimize for user value
- Right balance is key

8 DESIGNING FOR FAILURE

Accept that failures will happen. Build systems that anticipate, tolerate, and recover from those failures.

Key Points

- Expect failures
- Plan for recovery
- Test failures regularly
- Learn and improve

SUMMARY

RELIABILITY	AVAILABILITY	DURABILITY	RESILIENCE	FAULT TOLERANCE	HIGH AVAILABILITY	RELIABILITY VS PERFORMANCE	DESIGN FOR FAILURE
Do the right thing consistently.	Be there when users need you.	Protect data over time.	Recover fast and adapt.	Keep working when things fail.	Minimize downtime with smart design.	Balance speed with stability.	Failures are certain. Resilience is a choice.

"GOOD SYSTEMS HANDLE THE EXPECTED. GREAT SYSTEMS HANDLE THE UNEXPECTED." — SRE PRINCIPLE

VERIQTA
ENGINEERING EXCELLENCE

3. SERVICE LEVEL INDICATORS, SLIs

Measure what matters. Improve what you measure.

YOU CAN'T IMPROVE WHAT YOU DON'T MEASURE.

WHAT IS AN SLI?

A Service Level Indicator (SLI) is a measurable quantitative indicator that reflects the performance or reliability of a service as experienced by users.

Key Points

- An SLI measures actual service behavior.
- It is the foundation for setting SLOs.
- Good SLIs are user-focused and meaningful.
- Track a small number of important SLIs.

COMMON TYPES OF SLIs

AVAILABILITY INDICATORS	Measure the proportion of successful responses over time.
LATENCY INDICATORS	Measure how long it takes to service a user request.
ERROR RATE	Measure the proportion of failed requests.
THROUGHPUT	Measure the rate of successful requests processed.
CORRECTNESS	Measure whether the service returns the correct and expected result.
FRESHNESS	Measure how up-to-date the data or content is.

SLI CATEGORIES EXPLAINED

AVAILABILITY	Is the service available when users need it?	Example: 99.9% of successful responses over 30 days.
LATENCY	How fast is the service responding?	Example: 95th percentile latency < 300ms.
ERROR RATE	How often do requests fail?	Example: Error rate < 0.1% of requests.
THROUGHPUT	How many requests can the service handle?	Example: 10,000 successful requests per minute.
CORRECTNESS	Are users getting the right results?	Example: 99.95% of results are correct.
FRESHNESS	How current is the data users see?	Example: Data updated within last 60 seconds.

SELECTING MEANINGFUL SLIs

- Start with user experience and business objectives.
- Choose SLIs that reflect real user needs.
- Focus on a small set (3–5) of critical SLIs.
- Ensure SLIs are measurable with available data.
- Avoid vanity metrics that don't impact users.
- Review and iterate as your service evolves.

MEASURING USER EXPERIENCE

- Measure from the user's perspective (not just systems).
- Use real user monitoring (RUM) and synthetic checks.
- Track SLIs across key user journeys.
- Measure in different regions and devices.
- Understand impact, not just technical metrics.
- Combine quantitative data with user feedback.

EXAMPLES OF GOOD SLIs

AVAILABILITY	LATENCY	ERROR RATE	THROUGHPUT	CORRECTNESS	FRESHNESS
% of HTTP requests that return a success status (200–299) over a time window.	95th percentile latency of API responses.	% of HTTP requests that result in 4xx or 5xx errors.	Successful requests processed per minute.	% of transactions with correct expected results.	Time since data was last successfully updated.
99.95% over 30 days	P95 < 300ms over 30 days	< 0.1% over 30 days	10,000 RPM sustained	99.95% accuracy	< 60 sec max delay

GREAT SLIs REFLECT REAL USER EXPERIENCE, DRIVE BETTER DECISIONS, AND BUILD RELIABLE SERVICES.

VERIQTA
ENGINEERING EXCELLENCE

4. SERVICE LEVEL OBJECTIVES, SLOs

Define the reliability you promise. Deliver the reliability users expect.

AN SLO IS A TARGET. A PROMISE YOU INTEND TO KEEP.

WHAT IS AN SLO?

A Service Level Objective (SLO) is a target for an SLI over a specific time window. It represents the level of performance you aim to achieve.

Key Points

- It is based on actual user experience (SLIs).
- It is a target, not a guarantee.
- It drives engineering and business decisions.
- It is the foundation for error budgets.

SETTING RELIABILITY TARGETS

- Understand user needs and business goals.
- Analyze historical performance.
- Know the impact of downtime or errors.
- Balance reliability with cost and complexity.
- Align with capacity, architecture, and team maturity.
- Start realistic, then improve over time.
- Document and communicate clearly.
- Review and adjust regularly.

SLI AND SLO RELATIONSHIP

SLI — MEASURES WHAT IS HAPPENING → **SLO** — DEFINES WHAT WE AIM FOR → **ERROR BUDGET** — MEASURES HOW MUCH WE CAN RISK

Example: Availability SLI (Successful requests / Total requests)	SLO Target	Error Budget
	99.9% over 30 days	0.1% allowed over 30 days

If the SLI stays within the SLO target, users are happy.
If it goes over, the error budget is consumed.

CHOOSING REALISTIC TARGETS

- Use real data, not goals you hope to reach.
- Consider user expectations and industry standards.
- Evaluate cost, complexity, and reliability trade-offs.
- Avoid targets that require unsustainable effort.
- Improve gradually and consistently.

GOOD TARGETS

- Are challenging but achievable
- Can be maintained reliably
- Provide room for innovation
- Align with user impact

99.9% vs 99.99% AVAILABILITY

	99.9% (Three Nines)	99.99% (Four Nines)
Availability	99.9%	99.99%
Downtime Allowed	~8.76 hours / month	~52.56 minutes / month
Downtime Allowed	~3.65 days / year	~5.26 hours / year
Use Case	Internal tools, non-critical systems	Customer-facing, mission-critical systems
Cost & Complexity	Lower	Higher

Higher reliability requires more engineering, automation, redundancy, and operational maturity.

ROLLING MEASUREMENT WINDOWS

SLOs are evaluated over rolling time windows (e.g., 30 days).

- A rolling window includes recent data at all times.
- Provides a more accurate view than fixed periods.
- Smooths out short-term spikes or dips.
- Common windows: 7 days, 30 days, 90 days.

SLO COMPLIANCE

Track SLI against SLO target continuously.

- If SLI meets target = Compliant
- If SLI is below target = Non-Compliant
- Non-compliance consumes your error budget.
- Long-term non-compliance requirse action.

PRODUCTION EXAMPLES

WEB SERVICE AVAILABILITY
SLI: Successful HTTP requests
SLO: 99.9% availability
Window: 30 days
Business Impact:
Users can access the service reliably.

API LATENCY
SLI: 95th percentile latency
SLO: < 300ms
Window: 30 days
Business Impact:
Fast responses improve user satisfaction.

ERROR RATE
SLI: HTTP 5xx error rate
SLO: < 0.1%
Window: 30 days
Business Impact:
Fewer errors = more successful user actions.

CHECKOUT SUCCESS
SLI: Successful checkouts
SLO: 99.5%
Window: 7 days
Business Impact:
Directly affects revenue and user trust.

DATA FRESHNESS
SLI: Data updated within 5 minutes
5 minutes
SLO: 99%
Window: 24 hours
Business Impact:
Users see current and accurate information.

DEFINE CLEAR SLOS. MEASURE REAL USER EXPERIENCE. RESPECT THE ERROR BUDGET. DELIVER RELIABILITY. EARN TRUST.

VERIQTA
ENGINEERING EXCELLENCE

5. SLAs AND ERROR BUDGETS

Balance reliability goals with innovation and speed.

RELIABILITY IS A PROMISE. ERROR BUDGETS ARE FREEDOM WITH DISCIPLINE.

SERVICE LEVEL AGREEMENTS (SLAs)

An SLA is a contract between a service provider and customer that defines the level of service that will be delivered.

Key Points

- Legal or business commitment
- Defines consequences if service levels are not met
- Typically customer-facing
- Based on SLOs internally

SLO vs SLA

SLO (Internal)	SLA (External)
Engineering goal	Customer promise
Based on SLIs	Based on business commitment
Used to manage reliability	Used in contracts
Has error budget	No error budget exposed
May change over time	Harder to change
Internal teams use it	Customer sees it

VS

ERROR BUDGET FUNDAMENTALS

An error budget is the amount of unreliability you can afford while still meeting your SLO.

Key Concepts

- 100% - SLO = Error Budget
- Budget is spent when users experience failures
- Encourages risk-taking and innovation
- Protects reliability by limiting risky changes

CALCULATING ERROR BUDGETS

Error Budget = (1 - SLO Target) × Time Period

Example: SLO = 99.9% Availability

Time Period	Allowed Downtime	Error Budget
1 Day	1.44 minutes	0.1%
7 Days	10.08 minutes	0.1%
30 Days	43.2 minutes	0.1%
1 Year	8.76 hours	0.1%

Higher SLOs (e.g., 99.99%) = Smaller error budgets = Less room for failure.

ERROR BUDGET CONSUMPTION

Error budgets are consumed when the SLI falls below the SLO target.

How Budget Gets Consumed

- Service outages
- High latency
- Increased error rates
- Degraded performance
- User-impacting incidents

BURN RATES

Burn rate shows how fast you are using your error budget.

Burn Rate = Budget Consumed / Time

Common Burn Rate Thresholds

< 1x	Healthy (using budget slowly)
1x – 2x	Caution (watch closely)
2x – 5x	Warning (take action)
> 5x	Critical (immediate action)

Multi-window burn rate alerts help detect fast and slow burn situations.

RELEASE DECISIONS BASED ON ERROR BUDGET

PLENTY OF BUDGET	> 50% budget remaining Safe to release and innovate.
LOW BUDGET	20% – 50% remaining Release with caution. Increase testing.
BUDGET EXHAUSTED	= 0% remaining Stop releases. Focus on stability.

ERROR BUDGET POLICIES

- **STOP THE LINE:** No releases when budget is exhausted.
- **REDUCE RISK:** Reduce change rate as budget decreases.
- **AUTOMATIC ACTIONS:** Trigger controls based on burn rate.
- **PROTECT USER TRUST:** Prioritize reliability over features.
- **TRANSPARENT RULES:** Make policies clear to all teams.
- **REVIEW REGULARLY:** Adjust policies as service and org mature.
- **SHARE OWNERSHIP:** Everyone is responsible for reliability.

ERROR BUDGET LIFE CYCLE

1. **DEFINE SLO** — Set a clear reliability target.
2. **CALCULATE BUDGET** — Determine how much unreliability is allowed.
3. **MONITOR SLI** — Track real performance continuously.
4. **CONSUME BUDGET** — Failures consume the error budget.
5. **TAKE ACTION** — Adjust releases and mitigations.
6. **RESET WINDOW** — Budget resets after the time window.

Error budgets enable high reliability without sacrificing innovation. Use them wisely.

GOOD ENGINEERING IS ABOUT MANAGING RISK. ERROR BUDGETS MAKE THAT RISK VISIBLE AND ACTIONABLE. — SRE PRINCIPLE

VERIQTA
ENGINEERING EXCELLENCE

6. OBSERVABILITY FOR SRE

Full visibility. Faster insights. Better reliability.

YOU CAN'T IMPROVE WHAT YOU CAN'T OBSERVE.

THE FIVE PILLARS OF OBSERVABILITY

METRICS

Quantitative measurements over time. Used to understand system behavior and health.

Examples
- CPU usage
- Request rate
- Error rate
- Latency (p95)

LOGS

Discrete records of events that happened. Provide context and details.

Examples
- Application logs
- Access logs
- Audit logs
- Error logs

TRACES

Follow a request as it travels across services. Show latency and dependencies.

Examples
- Request trace
- Span timing
- Service map
- Call hierarchy

EVENTS

Notable things that happen in the system. Useful for awareness and automation.

Examples
- Deployments
- Alerts
- User signups
- Feature flags

CORRELATION

Connect signals from metrics, logs, traces, and events to understand the full picture.

Examples
- Trace ID in logs
- Metric + log correlation
- Alert linked to incident
- User journey correlation

TELEMETRY PIPELINES

Collect → Process → Store → Visualize → Act

Collect	Process	Store	Visualize	Act
Apps, hosts, infra, services	Parse, filter, enrich, batch	Time series DB, log store, trace DB	Dashboards, explorers, maps	Alerts, automation, tickets

Key Tools

Prometheus · Loki · Jaeger · OpenTelemetry · Grafana

OBSERVABILITY vs MONITORING

MONITORING	OBSERVABILITY
Tells you WHAT is wrong	Helps you understand WHY it is wrong
Reactive (detects known issues)	Proactive (explore unknown issues)
Based on predefined metrics and alerts	Based on rich telemetry and correlation
Answers specific questions	Asks any question about system behavior
Focus on infrastructure and uptime	Focus on user experience and reliability

BUILDING PRODUCTION VISIBILITY

- [x] Instrument everything that matters.
- [x] Use consistent naming and labels.
- [x] Capture RED or USE metrics for every service.
- [x] Include business metrics and SLIs.
- [x] Propagate trace context across all services.
- [x] Centralize logs with structured formats.
- [x] Correlate signals using IDs (trace, request, user).
- [x] Create dashboards that answer real questions.
- [x] Test your telemetry in production.

OBSERVABILITY-DRIVEN TROUBLESHOOTING

1. Define the scope and impact.
2. Check SLIs and dashboards (spot the symptom).
3. Drill down with metrics (quantify the issue).
4. Correlate with logs (find the context).
5. Follow traces (find the root service and latency source).
6. Identify recent changes or events.
7. Form a hypothesis and validate.
8. Resolve, then improve to prevent recurrence.

FROM SIGNALS TO INSIGHT

Collect Signals	Process & Correlate	Store	Visualize	Alert & Automate	Improve Reliability
(Metrics, Logs, Traces, Events)	(Enrich, Filter, Correlate IDs)	(Reliable, Scalable, Cost-Effective)	(Dashboards, Maps, Explore)	(Detect, Notify, Remediate)	(Learn, Optimize, Automate)

Great observability turns data into action and action into reliability.

OBSERVABILITY ISN'T A DESTINATION. IT'S A CONTINUOUS ENGINEERING PRACTICE. — SRE PRINCIPLE

PAGE 7

VERIQTA
ENGINEERING EXCELLENCE

SRE HANDBOOK

7. GOLDEN SIGNALS AND MONITORING STRATEGIES

FOCUS ON WHAT MATTERS. DETECT ISSUES EARLY. PROTECT USER EXPERIENCE.

GOOD MONITORING HELPS YOU FIX PROBLEMS BEFORE USERS FEEL THEM.

THE FOUR GOLDEN SIGNALS

1. LATENCY

The time it takes to serve a request.
High latency = slow experience.

KEY POINTS

- Track percentiles (p50, p90, p95, p99)
- Monitor end-to-end latency
- Set SLOs based on user needs

2. TRAFFIC

The number of requests or transactions over time.
Traffic changes impact capacity.

KEY POINTS

- Requests per second (RPS)
- Monitor trends and patterns
- Detect anomalies early

3. ERRORS

The rate of failed requests.
Errors impact reliability and user trust.

KEY POINTS

- Error rate = errors / requests
- Track by type and severity
- Monitor error budgets

4. SATURATION

How full your resources are.
High saturation leads to failures.

KEY POINTS

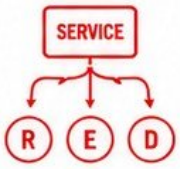
- CPU, memory, disk, queue depth
- Monitor headroom
- Plan capacity proactively

RED METHOD (FOR SERVICES)

SIGNAL		MEASURES
R	Rate	Number of requests per second
E	Errors	Number of failed requests
D	Duration	Time to service requests (latency)

USE RED FOR:

- Your application services
- APIs and microservices
- Anything you build

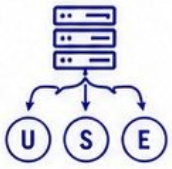


USE METHOD (FOR RESOURCES)

SIGNAL		MEASURES
U	Utilization	How busy the resource is
S	Saturation	How full the resource queue is
E	Errors	Error rate of the resource

USE USE FOR:

- Infrastructure components
- Databases, queues, caches
- Systems you depend on



MONITORING SERVICE HEALTH

- Is the service up and reachable?
- Is it performing within SLO?
- Are dependencies healthy?
- Is error rate within budget?
- Is latency within target?
- Is traffic normal?
- Is capacity sufficient?
- Are changes causing issues?
- Are users achieving their goals?

HEALTH = AVAILABILITY + PERFORMANCE + RELIABILITY + USER SUCCESS

USER-CENTRIC MONITORING

- Measure what users care about.
- Track user journeys, not just systems.
- Synthetic monitoring for key flows.
- Real User Monitoring (RUM).
- Monitor from real user locations.
- Focus on outcomes, not components.
- Map metrics to user experience.

IF USERS ARE HAPPY, YOUR MONITORING IS WORKING.

INFRASTRUCTURE vs APPLICATION SIGNALS

INFRASTRUCTURE SIGNALS

- CPU usage
- Memory usage
- Disk I/O and space
- Network throughput
- Queue length
- System errors
- Host availability

VS

APPLICATION SIGNALS

- Request rate
- Latency percentiles
- Error rate by type
- Business metrics
- Dependency latency
- Feature usage
- User behavior

COMBINE BOTH FOR FULL VISIBILITY AND FAST TROUBLESHOOTING.

MONITORING STRATEGIES

1. BASELINE

Understand normal behavior first.

- Establish baselines
- Use historical data
- Update regularly

2. DETECT

Detect anomalies quickly.

- Alert on deviation
- Use smart thresholds
- Reduce noise

3. ALERT

Alert the right people at the right time.

- Actionable alerts
- Include runbooks
- Escalate properly

4. RESPOND

Respond fast and minimize impact.

- Triage quickly
- Communicate clearly
- Restore service

5. LEARN

Learn and improve continuously.

- Postmortems
- Update dashboards
- Refine alerts

6. PREVENT

Prevent issues before they happen.

- Capacity planning
- Chaos testing
- Continuous improvement

THE GOAL OF MONITORING IS NOT MORE DATA. IT IS BETTER INSIGHT. — SRE PRINCIPLE

PAGE 8

VERIQTA

ENGINEERING EXCELLENCE

SRE HANDBOOK

8. ALERTING ENGINEERING

RIGHT ALERT. RIGHT PEOPLE. RIGHT TIME.

BETTER ALERTS = FASTER RESOLUTION + HAPPIER TEAMS

GOOD ALERTS DRIVE ACTION.
BETTER ALERTS DRIVE RELIABILITY.
LESS NOISE.
MORE IMPACT.

1. SYMPTOMS vs CAUSES

SYMPTOM (What you see)	CAUSE (What matters)
High CPU	Memory leak
500 errors	Dependency failure
High latency	Database saturation

VS

Alert on causes, not just symptoms.
Symptoms change. Causes are root problems.

2. ACTIONABLE ALERTS

- Tell you what is wrong
- Tell you where it is
- Tell you how bad it is
- Suggest what to do
- Link to runbooks / docs
- Drive a response

If an alert doesn't drive action, it's only noise.

3. ALERT THRESHOLDS

- Set thresholds based on real data
- Use percentiles, not averages
- Avoid static thresholds where possible
- Use dynamic or adaptive thresholds for variability
- Review and tune regularly

Bad thresholds = noisy alerts or missed incidents.

4. SLO-BASED ALERTING

SLO

- Align alerts with your SLOs
- Alert when you are at risk of violating the SLO
- Focus on user impact, not internal metrics
- Connect alerts to error budget

SLOs tell you what to achieve.
Alerts tell you when you're at risk.

5. MULTI-WINDOW BURN-RATE ALERTS

Detect fast burns early.
Detect slow burns before they become problems.

BURN RATE	SHORT WINDOW	LONG WINDOW	ACTION
Critical	> 14x	> 14x	Page Now
Warning	> 6x	> 1x	Investigate
Info	> 1x	> 0.5x	Monitor

Multi-window alerts balance speed and signal quality.

6. ALERT SEVERITY

Severity

CRITICAL	User impact is severe Immediate action required Page the on-call
WARNING	Degrading service Potential future impact Investigate soon
INFO	Informational only No immediate action

Correct severity = correct response.

7. WARNING vs CRITICAL

WARNING	CRITICAL
Early signal of a potential issue	User impact is real or imminent
System is working, but degrading	SLO is burning too fast
Allows time to investigate	Immediate response required
Example: Latency high but within SLO	Example: Error rate > SLO budget

VS

Warning gives time. Critical demands action.

8. ALERT ROUTING

- Send alerts to the right team
- Use routing rules by service, severity, environment
- Integrate with Pager, Slack, Email, SMS
- Escalate when no one acknowledges
- Include runbook links and dashboard links

Right alert. Right person. Right channel. Right time.

9. REDUCING ALERT FATIGUE

Eliminate Noise
Remove unused alerts. Avoid alerting on everything.

Group & Aggregate
Group related alerts. Alert on patterns, not single events.

Use Proper Thresholds
Tune thresholds to real behavior and baselines.

Correlate Alerts
Reduce duplicates. Show the full context.

Review Regularly
Review alert rules, false positives, and usefulness.

Empower Teams
Give teams ownership of their alerts.

FEWER, BETTER ALERTS = FASTER RESPONSE + LOWER STRESS + HIGHER RELIABILITY

VERIQTA

ENGINEERING EXCELLENCE

9. INCIDENT MANAGEMENT

RAPID RESPONSE. CLEAR COMMUNICATION. LASTING IMPROVEMENT.

A WELL-MANAGED INCIDENT IS A LEARNING OPPORTUNITY THAT MAKES SYSTEMS STRONGER.

INCIDENT LIFECYCLE

1 DETECTION	2 TRIAGE	3 SEVERITY CLASSIFICATION	4 ACKNOWLEDGE & ASSIGN	5 INVESTIGATE & DIAGNOSE	6 MITIGATION	7 RESOLUTION	8 RECOVERY	9 DOCUMENT & LEARN
Identify abnormal behavior or issues through alerts, users, or monitoring systems.	Validate the issue. Determine impact and scope. Avoid false urgency.	Classify based on impact and urgency. P1 (Critical) to P4 (Informational).	Acknowledge the incident. Assign an Incident Commander (IC).	Find the root cause. Collect data. Understand what is happening.	Implement actions to reduce impact or stop further degradation.	Fix the root cause. Restore normal functionality. Verify the fix.	Ensure full system recovery. Monitor closely to confirm stability.	Document the incident. Conduct post-incident review and learn.

KEY ROLES IN INCIDENT MANAGEMENT

INCIDENT COMMANDER (IC)	Owns the incident. Makes key decisions. Coordinates the response.
COMMUNICATOR	Manages all internal and external communication. Keeps stakeholders informed.
Scribe / Documenter	Documents timeline, decisions, actions, and updates. Maintains incident log.
ENGINEERS / RESPONDERS	Investigate, diagnose, and implement fixes. Provide technical updates.
STAKEHOLDERS	Product, support, leadership, and customers affected or interested.

COMMUNICATION PRINCIPLES

- **BE CLEAR:** Share simple, accurate updates.
- **BE TIMELY:** Communicate early and often.
- **BE TRANSPARENT:** Share what you know. Say what you don't know.
- **BE CONSISTENT:** Align all messages.
- **BE EMPATHETIC:** Users are impacted. Show you care.

GOOD COMMUNICATION REDUCES ANXIETY AND BUILDS TRUST.

6. MITIGATION

- Act fast to reduce customer impact.
- Use safe, low-risk actions first.
- Focus on containing the problem.
- It's okay to be temporary.
- Update stakeholders on actions taken.

GOAL: STOP THE BLEEDING.

7. RESOLUTION

- Fix the root cause, not just symptoms.
- Verify the solution in production.
- Test before declaring resolved.
- Confirm with monitoring and users.
- Communicate resolution clearly.

GOAL: RESTORE NORMAL SERVICE.

8. RECOVERY

- Monitor system health closely.
- Look for regressions or side effects.
- Validate all systems are stable.
- Gradually return to normal operations.
- Confirm with all key stakeholders.

GOAL: ENSURE STABILITY AND CONFIDENCE.

9. INCIDENT DOCUMENTATION AND LEARNING

RECORD EVERYTHING
- Timeline of events
- Decisions made
- Actions taken
- People involved
- Systems affected

POST-INCIDENT REVIEW (PIR)
- What happened?
- Why did it happen?
- What went well?
- What didn't go well?
- What can we improve?

ACTION ITEMS
- Create specific actions
- Assign owners
- Set due dates
- Track to completion
- Verify improvement

SHARE AND IMPROVE
- Share learnings across teams
- Update runbooks and docs
- Improve alerts and monitoring
- Strengthen systems
- Prevent future incidents

MEASURE IMPACT
- MTTD (Detect)
- MTTA (Acknowledge)
- MTTR (Resolve)
- Customer Impact
- Incident Frequency

EVERY INCIDENT IS A CHANCE TO IMPROVE OUR SYSTEMS, PROCESSES, AND PEOPLE.

VERIQTA
ENGINEERING EXCELLENCE

10. PRODUCTION TROUBLESHOOTING

A SYSTEMATIC APPROACH. FASTER ANSWERS. BETTER OUTCOMES.

GOOD TROUBLESHOOTING FINDS THE CAUSE.
GREAT TROUBLESHOOTING PREVENTS IT NEXT TIME.

THE STRUCTURED TROUBLESHOOTING FLOW

1. **ESTABLISH IMPACT** — Understand the impact on users, business, and SLIs/SLOs.
2. **BUILD A TIMELINE** — Create a timeline of events. When did it start? What changed?
3. **CHECK METRICS** — Review golden signals and key SLIs. Identify what's abnormal.
4. **ANALYZE LOGS** — Search logs for errors, exceptions, patterns, and correlations.
5. **TRACE THE REQUEST** — Use distributed tracing to follow the request end-to-end.
6. **INVESTIGATE DEPENDENCIES** — Check downstream services, databases, queues, APIs, and third parties.
7. **REVIEW CHANGES** — Look at recent deployments, config changes, and feature flags.
8. **FORM HYPOTHESIS & TEST** — Form a hypothesis for the root cause. Test it with data, not guesses.
9. **ISOLATE ROOT CAUSE** — Confirm the true root cause with evidence. Avoid multiple theories.
10. **RESOLVE & VALIDATE** — Implement the fix. Validate with data. Monitor to ensure stability.

1. ESTABLISH IMPACT

- Who is affected?
- What features are down?
- What is the business impact?
- Check SLO/SLA violations.
- Determine urgency.

FOCUS ON WHAT MATTERS MOST.

2. BUILD A TIMELINE

- Note when the issue started.
- Map significant events.
- Include deployments, config changes, incidents.
- Use UTC timestamps.

A CLEAR TIMELINE REVEALS PATTERNS.

3. CHECK METRICS

- Check latency, traffic, errors, and saturation.
- Compare with baseline.
- Use dashboards and alerts.
- Identify the scope.

METRICS SHOW THE BIG PICTURE.

4. ANALYZE LOGS

- Search for errors and exceptions.
- Filter by time, service, and correlation ID.
- Look for patterns and trends.
- Don't just skim—investigate.

LOGS TELL YOU WHAT HAPPENED.

5. TRACE THE REQUEST

- Follow the request across services.
- Identify slow calls and failures.
- Check spans, tags, and annotations.
- Understand end-to-end behavior.

TRACES SHOW WHERE IT HAPPENED.

6. INVESTIGATE DEPENDENCIES

- Check health of dependent services.
- Validate database performance.
- Inspect queues, caches, APIs.
- Verify third-party status pages.

FAILURES OFTEN COME FROM SOMEWHERE ELSE.

7. REVIEW CHANGES

- Check recent deployments.
- Review config and infra changes.
- Inspect feature flags.
- Compare versions and settings.

RECENT CHANGES ARE COMMON CULPRITS.

8. HYPOTHESIS-DRIVEN DEBUGGING

- State a clear hypothesis.
- Test with experiments or data.
- Prove or disprove quickly.
- Avoid random searching.

TEST. DON'T GUESS. PROVE. DON'T ASSUME.

9. ROOT CAUSE ISOLATION

- Confirm the single true cause.
- Gather concrete evidence.
- Rule out other possibilities.
- Document why it happened.

FIND THE CAUSE, NOT THE BLAME.

10. RESOLVE & VALIDATE

- Implement the fix.
- Validate in production.
- Monitor metrics and logs.
- Confirm stability and close.

FIX IT RIGHT. KEEP IT STABLE.

TROUBLESHOOTING MINDSET

- DATA OVER ASSUMPTIONS
- SIMPLE FIRST, THEN COMPLEX
- BE SYSTEMATIC AND PATIENT
- COMMUNICATE FREQUENTLY
- LEARN AND IMPROVE

QUICK CHECKLIST

- [x] Is the impact clear?
- [x] Do you have a timeline?
- [x] What do the metrics say?
- [x] What do the logs say?
- [x] What do the traces show?
- [x] Are dependencies healthy?
- [x] Any recent changes?
- [x] What is your hypothesis?
- [x] What is the root cause?
- [x] Is the issue resolved and validated?

GREAT TROUBLESHOOTING = STRUCTURE + DATA + COMMUNICATION + EXPERIENCE

VERIQTA
ENGINEERING EXCELLENCE

11. POSTMORTEMS AND LEARNING FROM FAILURE

BLAMELESS. FACT-BASED. ACTION-ORIENTED. FUTURE-FOCUSED.

FAILURES WILL HAPPEN. LEARNING FROM THEM IS HOW WE GET BETTER.

THE POSTMORTEM PRINCIPLES

BLAMELESS
We focus on systems, not people.

FACT-BASED
We use data, not opinions or assumptions.

ACTION-ORIENTED
We create actions that prevent recurrence.

LEARN AND SHARE
We share learnings across teams to make everyone stronger.

CONTINUOUSLY IMPROVE
We review actions and improve our systems continuously.

THE POSTMORTEM CONTENT FRAMEWORK

1. BLAMELESS POSTMORTEMS

- No blame. No shame.
- Focus on the system.
- People make mistakes.
- Systems should be safe.

SAFETY ENABLES HONESTY.

2. INCIDENT TIMELINE

- When did it start?
- What happened?
- When did it escalate?
- When was it resolved?

A CLEAR TIMELINE CREATES CLARITY.

3. ROOT CAUSE

- What was the root cause?
- Why did it happen?
- Use "5 Whys" if needed.
- Be specific and factual.

FIND THE CAUSE, NOT THE SYMPTOM.

4. CONTRIBUTING FACTORS

- What conditions contributed?
- Complex interactions?
- System weaknesses?
- Process or tooling gaps?

MULTIPLE FACTORS RARELY ONE.

5. DETECTION GAPS

- Why didn't we detect earlier?
- Missing alerts?
- Poor visibility?
- Monitoring gaps?

EARLIER DETECTION REDUCES IMPACT.

6. RESPONSE GAPS

- What slowed us down?
- Runbook gaps?
- Communication issues?
- Escalation delays?

FASTER RESPONSE REDUCES IMPACT.

7. CORRECTIVE ACTIONS

- What will we fix?
- Short-term actions.
- Addresses the root cause.
- Clear and specific.

FIX THE SYSTEM, NOT JUST THE ISSUE.

8. PREVENTIVE ACTIONS

- How do we prevent it?
- Process improvements.
- Tooling and automation.
- Monitoring enhancements.

PREVENTION IS THE REAL GOAL.

9. OWNERSHIP AND DEADLINES

- Who owns each action?
- What is the deadline?
- Track progress.
- Ensure accountability.

OWNERSHIP DRIVES ACCOUNTABILITY.

10. LEARNING CULTURE

- Share lessons learned.
- Improve runbooks.
- Educate and enable teams.
- Make the system stronger.

WE LEARN TOGETHER. WE IMPROVE TOGETHER.

POSTMORTEM CHECKLIST

- Incident timeline is complete
- Root cause is identified
- Contributing factors are clear
- Actions are specific and measurable
- Owners and deadlines set
- Shared with relevant teams
- Follow-up and tracking in place

EVERY POSTMORTEM IS AN INVESTMENT IN A MORE RELIABLE FUTURE.

VERIQTA

ENGINEERING EXCELLENCE

12. TOIL AND SRE AUTOMATION

LESS TOIL. MORE AUTOMATION. MORE TIME FOR WHAT MATTERS.

AUTOMATE TO ELIMINATE TOIL. ENGINEER THE PLATFORM. FREE PEOPLE TO INNOVATE.

CORE CONCEPTS

1. WHAT TOIL IS

Manual, repetitive, reactive work with low or no enduring value.

2. SOURCES OF OPERATIONAL TOIL

- Manual changes
- Alert noise
- Firefighting
- Manual deployments
- Reporting
- Access management

3. MEASURING TOIL

- Track time spent on toil
- Count repetitive tasks
- Use surveys and logs
- Measure impact on velocity

4. TOIL BUDGETS

- Allocate a % of capacity for toil
- Make toil visible
- Drive automation investment

5. MANUAL OPERATIONS

- SSH and manual scripts
- Manual prod changes
- Copy-paste work
- Ad-hoc troubleshooting
- Spreadsheet tracking

FROM TOIL TO AUTOMATION

6. AUTOMATION OPPORTUNITIES

- Identify high-frequency tasks
- Focus on repeatable steps
- Prioritize by impact and effort
- Automate small, iterate fast

AUTOMATE WHAT HURTS THE MOST, HAPPENS THE MOST.

7. RUNBOOK AUTOMATION

- Convert runbooks to code
- Use scripts and tools
- Standardize procedures
- Version control runbooks

FROM DOCUMENTS TO EXECUTABLE WORKFLOWS.

8. SELF-HEALING SYSTEMS

- Detect issues automatically
- Take safe, predefined actions
- Recover without human intervention
- Escalate when needed

BUILD SYSTEMS THAT HEAL THEMSELVES.

9. REMOVING REPETITIVE WORK

- Eliminate manual approvals where safe
- Replace handoffs with automation
- Reduce alert noise
- Improve tooling

EVERY MINUTE SAVED IS ENGINEERING TIME GAINED.

10. ENGINEERING vs OPERATIONAL WORK

OPERATIONAL (TOIL)	ENGINEERING (VALUE)
Reactive	Proactive
Manual	Automated
Repetitive	Creative
Short term	Long term
Firefighting	Problem solving

MOVE LEFT: SHIFT WORK FROM TOIL TO VALUE.

TOIL ELIMINATION FRAMEWORK

IDENTIFY → **MEASURE** → **PRIORITIZE** → **AUTOMATE** → **VALIDATE** → **ITERATE**

- **IDENTIFY**: Find repetitive, manual, and low-value tasks.
- **MEASURE**: Quantify time, frequency, and impact.
- **PRIORITIZE**: Pick tasks with high impact and high frequency.
- **AUTOMATE**: Build tools, scripts, and integrations to automate.
- **VALIDATE**: Ensure automation is reliable and safe.
- **ITERATE**: Continuously improve and automate more.

THE GOAL

ELIMINATE TOIL. IMPROVE RELIABILITY. INCREASE VELOCITY. EMPOWER ENGINEERS. BUILD BETTER SYSTEMS. DELIVER MORE VALUE.

VERIQTA
ENGINEERING EXCELLENCE

13. ON-CALL ENGINEERING

BE PREPARED. RESPOND EFFECTIVELY. RESTORE RELIABLY.

GREAT ON-CALL IS NOT ABOUT HEROES. IT'S ABOUT SYSTEMS THAT SUPPORT PEOPLE.

ON-CALL FOUNDATIONS

1. ON-CALL RESPONSIBILITIES

- Monitor alerts and systems
- Triage and acknowledge alerts
- Investigate and diagnose issues
- Communicate status updates
- Mitigate, restore, and verify
- Document and follow up

YOU OWN THE RESPONSE UNTIL IT IS RESOLVED.

2. ROTATIONS

- Fair and predictable schedules
- Follow follow-the-sun model
- Limit consecutive night shifts
- Include overlap time
- Respect time off

SUSTAINABLE SCHEDULES BUILD STRONG TEAMS.

3. ESCALATION POLICIES

- Clear escalation paths
- Define levels and owners
- Escalate by severity
- Time-based escalation
- No escalation = incident

ESCALATE EARLY. ESCALATE CLEARLY.

4. PAGING

- Alert routes to the right people
- Use severity-based paging
- Include context in alerts
- Avoid unnecessary pages
- Confirm acknowledgements

THE RIGHT ALERT. THE RIGHT PERSON. THE RIGHT TIME.

5. HANDOFFS

- Clear shift start and end
- Review open incidents
- Share context and updates
- Confirm understanding
- Document ongoing work

GOOD HANDOFFS PREVENT INCIDENTS.

ON-CALL EFFECTIVENESS

6. RUNBOOKS

- Step-by-step instructions
- Clear, simple, and tested
- Include commands & links
- Keep up to date
- Easy to find during incidents

GREAT RUNBOOKS SAVE TIME AND REDUCE MISTAKES.

7. PLAYBOOKS

- Solutions for common issues
- Standard troubleshooting paths
- Include rollback steps
- Define decision points
- Improve after incidents

PLAYBOOKS SPEED UP DECISIONS.

8. INCIDENT READINESS

- Know your services deeply
- Test runbooks regularly
- Run game days
- Review past incidents
- Prepare before incidents

READINESS IS THE BEST ON-CALL STRATEGY.

9. REDUCING NOISY ALERTS

- Fix alert flood at the source
- Set meaningful thresholds
- Aggregate and group alerts
- Remove alert duplicates
- Continuously tune alerts

LESS NOISE. MORE SIGNAL.

10. SUSTAINABLE ON-CALL OPERATIONS

- Protect rest and well-being
- Limit after-hours work
- Respect time off
- Balance load fairly
- Continuously improve

SUSTAINABLE TEAMS DELIVER RELIABLE SYSTEMS.

ON-CALL SUCCESS PRINCIPLES

OWN THE INCIDENT
Take ownership until it is fully resolved.

COMMUNICATE CLEARLY
Keep everyone informed with accurate updates.

FOCUS ON IMPACT
Solve the right problem. Reduce customer impact.

WORK AS A TEAM
Collaborate. Escalate. Ask for help.

LEARN AND IMPROVE
Every incident is an opportunity to improve.

ON-CALL IS A TEAM SPORT. SYSTEMS, PROCESSES, AND EMPATHY MAKE IT WORK.

VERIQTA

ENGINEERING EXCELLENCE

14. RESILIENCE AND FAILURE ENGINEERING

BUILD SYSTEMS THAT STAY AVAILABLE, EVEN WHEN THINGS FAIL.

RESILIENT SYSTEMS DON'T AVOID FAILURE. THEY EXPECT IT. THEY SURVIVE IT. THEY RECOVER FAST.

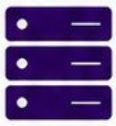
RESILIENCE PRINCIPLES OVERVIEW

1. FAILURE DOMAINS

- Isolate failures to prevent widespread impact.
- Use zones, regions, cells.
- Design blast-radius boundaries.

ISOLATE TO CONTAIN FAILURE.

2. REDUNDANCY



- Duplicate critical components.
- Multi-AZ, multi-region.
- No single point should cause total outage.

REMOVE SINGLE POINTS OF FAILURE.

3. GRACEFUL DEGRADATION

- Maintain core functionality under stress.
- Degrade non-essential features first.
- Communicate clearly to users.

SOMETHING IS BETTER THAN NOTHING.

4. TIMEOUTS

- Never wait forever.
- Set timeouts for all external calls.
- Fail fast and move on.

BOUND WAIT TIME. KEEP SYSTEMS FAST.

5. RETRIES

- Retry transient failures.
- Only for safe operations.
- Use limited attempts and proper delays.

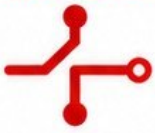
RETRY WISELY. AVOID STORMING.

6. EXPONENTIAL BACKOFF

- Increase wait time between retries.
- Reduce load on struggling services.
- Add jitter to avoid retry storms.

BACK OFF TO GIVE BREATH.

7. CIRCUIT BREAKERS



- Stop requests to failing services.
- Allow time for recovery.
- Prevent cascading failures.

FAIL FAST. RECOVER SMART.

8. BULKHEADS

- Isolate resources between components.
- Prevent one part from exhausting others.
- Use separate pools, queues, or containers.

CONTAIN FAILURE. PROTECT THE REST.

9. RATE LIMITING

- Limit requests per user or service.
- Prevent abuse and overload.
- Protect downstream dependencies.

CONTROL TRAFFIC. PROTECT SYSTEMS.

10. LOAD SHEDDING

- Drop non-critical work under heavy load.
- Protect critical paths and users.
- Shed early, not when the system collapses.

SHED LOAD EARLY. STAY AVAILABLE.

11. ELIMINATING SINGLE POINTS OF FAILURE

- Identify all single points.
- Redesign for removal.
- Validate with tests, chaos, and reviews.

NO SINGLE POINT. NO SINGLE FAILURE.

RESILIENCE IN ACTION

ANTICIPATE FAILURES
Assume things will break.

DESIGN FOR RESILIENCE
Use proven patterns and safeguards.

TEST UNDER FAILURE
Use chaos engineering and failure testing.

DETECT AND RESPOND
Observe, alert, and respond quickly.

RECOVER AND ADAPT
Heal fast. Learn every time.

CONTINUOUSLY IMPROVE
Make resilience a continuous practice.

RESILIENCE CHECKLIST

- Have we identified all failure domains?
- Do we have redundancy for critical components?
- Do we handle failures with graceful degradation?
- Are timeouts set for all external calls?
- Are retries safe, limited, and observable?
- Are we using exponential backoff with jitter?
- Do we have circuit breakers in place?
- Are bulkheads isolating critical resources?
- Are rate limits protecting our systems?
- Can we shed load when overwhelmed?
- Have we removed single points of failure?
- Do we test failure regularly?

RESILIENCE IS DESIGNED. RELIABILITY IS EARNED. TRUST IS DELIVERED.

VERIQTA

ENGINEERING EXCELLENCE

15. CAPACITY PLANNING AND PERFORMANCE

RIGHT CAPACITY. RIGHT TIME. RIGHT PERFORMANCE.

PLAN CAPACITY TODAY.
PERFORM TOMORROW.
PREPARE FOR PEAK.
DELIVER WITH CONFIDENCE.

CAPACITY PLANNING FUNDAMENTALS

1. TRAFFIC FORECASTING

- Analyze historical trends.
- Identify patterns (daily, weekly, seasonal).
- Account for business growth and events.

FORECAST ACCURATELY. PREPARE CONFIDENTLY.

2. RESOURCE UTILIZATION

- Monitor CPU, memory, disk, network.
- Track utilization over time.
- Understand normal vs abnormal.

YOU CAN'T IMPROVE WHAT YOU DON'T MEASURE.

3. SATURATION

- Know your system limits.
- Watch for sustained high utilization.
- Identify the point of degradation.

AT SATURATION, PERFORMANCE DIES.

4. HEADROOM

- Maintain healthy buffer above normal load.
- Ensure room for spikes and unexpected growth.
- Plan for at least 20-30% headroom.

HEADROOM TODAY. STABILITY TOMORROW.

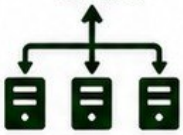
5. SCALING THRESHOLDS

- Define clear metrics to trigger scaling.
- Set thresholds based on utilization and latency.
- Avoid reactive scaling.

SCALE BEFORE USERS FEEL THE PAIN.

SCALING STRATEGIES

6. HORIZONTAL SCALING



- Add more instances.
- Distribute load.
- Improves availability and fault tolerance.

SCALE OUT. SHARE THE LOAD.

7. VERTICAL SCALING

- Increase power of existing instance.
- Simple but has limits.
- May require downtime.

SCALE UP. MORE POWER.

8. AUTOSCALING

- Automatically adjust capacity.
- Based on metrics and policies.
- Scale out and in as needed.

AUTOMATE SCALE. ELIMINATE GUESSWORK.

9. LOAD TESTING

- Simulate real traffic.
- Find capacity limits.
- Validate performance before production.

TEST LOAD. TRUST RESULTS.

10. PERFORMANCE BOTTLENECKS

- Identify slow components.
- CPU, memory, disk, network, database, external services.
- Fix the real constraints.

FIX THE BOTTLENECK. NOT THE SYMPTOM.

CAPACITY PLANNING FOR PEAK TRAFFIC

UNDERSTAND PEAK PATTERNS
Know when and why peaks happen.

ESTIMATE PEAK LOAD
Forecast peak traffic and resource needs.

PLAN CAPACITY
Ensure infrastructure can handle the load.

IMPLEMENT SCALING
Use autoscaling and elastic resources.

MONITOR & OPTIMIZE
Monitor during peaks. Optimize after peaks.

REVIEW & IMPROVE
Analyze, learn, and improve capacity strategy.

CAPACITY PLANNING CHECKLIST

- ☑ Do we understand our traffic patterns?
- ☑ Do we forecast growth and peak events?
- ☑ Are we monitoring key resources?
- ☑ Do we know our saturation points?
- ☑ Do we maintain enough headroom?
- ☑ Are our scaling thresholds defined?
- ☑ Do we have autoscaling in place?
- ☑ Have we load tested our systems?
- ☑ Are bottlenecks identified and fixed?
- ☑ Can we handle peak traffic?
- ☑ Do we regularly review capacity?
- ☑ Are we prepared for growth?

PLAN CAPACITY. TEST LIMITS. REMOVE BOTTLENECKS. DELIVER PERFORMANCE AT SCALE.

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

VERIQTA

ENGINEERING EXCELLENCE

16. SRE FOR KUBERNETES AND CLOUD PLATFORMS

RELIABLE PLATFORMS DELIVER VALUE. RESILIENT CLUSTERS EARN TRUST. SRE BUILDS BOTH.

BUILD RESILIENT PLATFORMS. AUTOMATE RELIABILITY. DELIVER CONTINUOUSLY.

KUBERNETES RELIABILITY FOUNDATIONS

1. KUBERNETES RELIABILITY

- Declarative desired state and self-healing.
- Controllers ensure continuous reconciliation.
- Design for failure, not perfection.

K8S IS BUILT TO RECOVER AUTOMATICALLY.

2. POD AND NODE FAILURES

- Pods are ephemeral.
- Nodes can fail anytime.
- Ensure workloads are rescheduled automatically.
- Use anti-affinity and multi-zone placement.

EXPECT FAILURES. DESIGN FOR CONTINUITY.

3. HEALTH PROBES

- Liveness probe: is the app alive?
- Readiness probe: is the app ready?
- Startup probe: avoid false restarts.

GOOD PROBES PREVENT BAD FAILURES.

4. REQUESTS AND LIMITS

- Requests ensure scheduling and QoS.
- Limits prevent noisy neighbor issues.
- Set based on real usage and testing.

RIGHT SIZING BOOSTS STABILITY AND COST.

5. AUTOSCALING

- HPA: scale pods by metrics.
- VPA: right-size requests.
- Cluster Autoscaler: adjust nodes.
- Combine for best results.

SCALE AUTOMATICALLY. STAY AVAILABLE.

RELIABILITY ENGINEERING PRACTICES

6. POD DISRUPTION BUDGETS (PDB)

- Protect availability during voluntary disruptions.
- Define minAvailable or maxUnavailable.
- PDBs + Deployments = safe updates.

DON'T DISRUPT MORE THAN YOU CAN AFFORD.

7. MULTI-ZONE ARCHITECTURE

- Spread across zones.
- Use topology spread constraints.
- Avoid single-zone dependencies.

DISTRIBUTE EVERYTHING. AVOID SINGLE ZONE RISK.

8. CLOUD SERVICE DEPENDENCIES

- Know your dependencies.
- Design for service throttling and outages.
- Implement retries, timeouts, and fallbacks.

DEPENDENCIES FAIL. PLAN FOR IT.

9. KUBERNETES OBSERVABILITY

- Metrics: Prometheus.
- Logs: Centralized logging.
- Traces: Distributed tracing.
- Events: Kubernetes events and alerts.

YOU CAN'T IMPROVE WHAT YOU CAN'T SEE.

10. CLUSTER FAILURE SCENARIOS

- Control plane failure.
- etcd failure.
- Network partition.
- DNS or CNI failure.

PRACTICE FAILURE. STAY PREPARED.

SRE WORKFLOW FOR KUBERNETES PLATFORMS

1. **OBSERVE** — Monitor metrics, logs, events, and traces continuously.
2. **DETECT** — Alert on real issues using meaningful thresholds.
3. **TRIAGE** — Identify impact, root cause, and blast radius.
4. **MITIGATE** — Take action using runbooks and automation.
5. **RESOLVE** — Restore service and validate reliability.
6. **LEARN** — Run postmortem and improve continuously.

KUBERNETES SRE CHECKLIST

- ☑ Are workloads multi-zone?
- ☑ Are health probes configured?
- ☑ Are requests and limits set?
- ☑ Is autoscaling enabled and tested?
- ☑ Do PDBs protect critical apps?
- ☑ Are dependencies resilient?
- ☑ Is observability complete?
- ☑ Are alerts actionable?
- ☑ Are runbooks up to date?
- ☑ Have we tested failure scenarios?
- ☑ Is capacity planned for peaks?
- ☑ Are we learning and improving?

DESIGN FOR FAILURE. ENGINEER FOR RECOVERY. OPERATE WITH CONFIDENCE.

VERIQTA

ENGINEERING EXCELLENCE

17. DEPLOYMENT RELIABILITY AND RELEASE ENGINEERING

RELEASE WITH CONFIDENCE.
VALIDATE CONTINUOUSLY.
ROLL BACK FAST.
DELIVER VALUE SAFELY.

REDUCE RISK. VALIDATE EARLY. DELIVER RELIABLY.

DEPLOYMENT AND RELEASE FUNDAMENTALS

1. DEPLOYMENT RISK

- Every change introduces risk.
- Risk = Impact × Probability.
- Reduce blast radius.
- Automate to reduce human error.

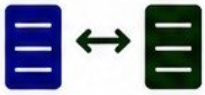
MEASURE RISK.
MANAGE RISK.
MINIMIZE RISK.

2. ROLLING DEPLOYMENTS

- Update instances gradually.
- Maintain availability.
- Monitor during rollout.
- Rollback if errors increase.

SMALL BATCHES.
CONTINUOUS AVAILABILITY.
QUICK ROLLBACK.

3. BLUE-GREEN DEPLOYMENTS



- Two identical environments.
- Switch traffic to green.
- Instant rollback to blue.
- Ideal for high-risk changes.

SWITCH TRAFFIC,
NOT INFRASTRUCTURE.

4. CANARY RELEASES

- Release to a small subset.
- Monitor key metrics.
- Increase gradually.
- Stop if metrics degrade.

TEST IN PRODUCTION
WITH LIMITED BLAST
RADIUS.

5. PROGRESSIVE DELIVERY

- Automate the rollout process.
- Progressive steps with automated gates.
- Pause, verify, proceed.
- Reduce risk continuously.

AUTOMATE PROGRESSION.
VERIFY AT EVERY STEP.

RELEASE ENGINEERING PRACTICES

6. FEATURE FLAGS

- Decouple deployment from release.
- Enable, disable, or target features.
- Safe experimentation.
- Reduce risk of change.

DEPLOY OFTEN.
RELEASE WHEN READY.

7. AUTOMATED VALIDATION

- Automated tests at every stage.
- Unit, integration, e2e, security, performance.
- Quality gates prevent bad releases.

AUTOMATE QUALITY.
SHIFT LEFT VALIDATION.

8. HEALTH CHECKS

- Check app, dependencies, and infrastructure.
- Use synthetic and real user checks.
- Validate after each step.

HEALTHY SYSTEMS
GET TRAFFIC.

9. ROLLBACK STRATEGIES

- Plan rollback before release.
- Keep previous version available.
- Automate rollback triggers.
- Practice rollback regularly.

ROLL BACK FAST.
MINIMIZE IMPACT.

10. REDUCING CHANGE FAILURE RATE

- Smaller changes.
- Automated testing.
- Progressive delivery.
- Observability and fast feedback.

SMALL CHANGES.
SMART VALIDATION.
SAFE DELIVERIES.

THE RELIABLE RELEASE PIPELINE

1. **PLAN** — Define change, assess risk, prepare rollout plan.
2. **BUILD** — Build artifact, run automated tests.
3. **VALIDATE** — Automated tests, security scans, quality gates.
4. **DEPLOY** — Choose strategy: rolling, blue-green, canary, or progressive.
5. **MONITOR** — Monitor metrics, logs, alerts, and user experience.
6. **VERIFY** — Validate health, SLOs, and business metrics.
7. **ROLLBACK IF NEEDED** — Automatic or manual rollback to last known good state.

RELEASE READINESS CHECKLIST

- ☑ Is the change scoped and approved?
- ☑ Are risks and rollback plan defined?
- ☑ Are automated tests passing?
- ☑ Are security scans clean?
- ☑ Are health checks configured?
- ☑ Is observability in place?
- ☑ Are alerts routed correctly?
- ☑ Is traffic routing strategy defined?
- ☑ Are canary thresholds configured?
- ☑ Are SLOs and error budgets healthy?
- ☑ Is documentation updated?
- ☑ Is the team and on-call informed?
- ☑ Is rollback tested and ready?
- ☑ Is post-release monitoring active?
- ☑ Is communication plan ready?
- ☑ Are success metrics defined?

RELEASE ENGINEERING IS HOW WE DELIVER CHANGE SAFELY, REPEATEDLY, AND RELIABLY.

VERIQTA

ENGINEERING EXCELLENCE

18. DISASTER RECOVERY AND BUSINESS CONTINUITY

PREPARE FOR DISASTER. RECOVER FAST. CONTINUE BUSINESS.

DISASTERS WILL HAPPEN. RESILIENCE IS A CHOICE. BE PREPARED.

DISASTER RECOVERY FUNDAMENTALS

1. DISASTER RECOVERY FUNDAMENTALS

- Plan for major disruptions.
- Protect people, data, systems, and operations.
- Define roles, runbooks, and communication.
- Align DR with business priorities.

PLAN TODAY. SURVIVE TOMORROW. THRIVE ALWAYS.

2. RTO (RECOVERY TIME OBJECTIVE)

- Maximum acceptable downtime.
- Defines speed of recovery.
- Set realistic and measurable targets.
- Varies by application criticality.

KNOW YOUR RTO. DESIGN TO MEET IT.

3. RPO (RECOVERY POINT OBJECTIVE)

- Maximum acceptable data loss.
- Defines how much data you can afford to lose.
- Measured in seconds, minutes, or hours.

EVERY SECOND COUNTS. DEFINE YOUR RPO.

4. BACKUP STRATEGY

- Follow 3-2-1 backup rule.
- Full, incremental, and differential backups.
- Encrypt backups.
- Store offsite or in another region.

GOOD BACKUPS SAVE BUSINESSES.

5. RESTORE TESTING

- Test restores regularly.
- Verify data integrity.
- Automate restore tests where possible.
- Document results and fix gaps.

UNTESTED BACKUPS ARE JUST HOPES.

DISASTER RECOVERY ARCHITECTURE & OPERATIONS

6. REGIONAL FAILURES

- Regions can fail.
- Design for regional outages.
- Isolate failures.
- Maintain availability during regional loss.

ASSUME FAILURE. BUILD RESILIENCE.

7. MULTI-REGION ARCHITECTURE

- Distribute across multiple regions.
- Active-active or active-passive.
- Replicate data and services.

DISTRIBUTE TO WITHSTAND DISRUPTION.

8. FAILOVER

- Switch traffic to healthy site.
- Automate failover where possible.
- Keep DNS and routing updated.

FAST FAILOVER. MINIMAL DOWNTIME.

9. DATA RECOVERY

- Restore data quickly and accurately.
- Use replication, snapshots, and backups.
- Verify consistency after restore.

DATA IS CRITICAL. RECOVER IT RIGHT.

10. DISASTER SIMULATIONS

- Run disaster simulations.
- Test people, process, and technology.
- Identify weaknesses and improve.

PRACTICE DISASTER. IMPROVE RECOVERY.

RECOVERY VALIDATION LIFECYCLE

1. **ASSESS RISK** — Identify threats, impacts, and dependencies.
2. **PLAN** — Define DR strategy, RTO, RPO, and recovery runbooks.
3. **IMPLEMENT** — Build infrastructure, replication, backups, and automation.
4. **TEST** — Test restores, failover, and runbooks.
5. **VALIDATE** — Validate results. Measure against RTO and RPO.
6. **IMPROVE** — Fix gaps and continuously improve.

DR READINESS CHECKLIST

- ☑ Do we know our RTO and RPO?
- ☑ Are backups automated and encrypted?
- ☑ Are backups stored offsite or in another region?
- ☑ Do we test restores regularly?
- ☑ Is our architecture multi-region ready?
- ☑ Is failover automated and tested?
- ☑ Are runbooks documented and updated?
- ☑ Do we monitor replication and backups?
- ☑ Have we run disaster simulations?
- ☑ Are communication plans tested?
- ☑ Is recovery validated against targets?
- ☑ Are we ready to recover today?

DISASTERS ARE UNPREDICTABLE. RECOVERY IS OPTIONAL. PREPARE. TEST. RECOVER.

VERIQTA

ENGINEERING EXCELLENCE

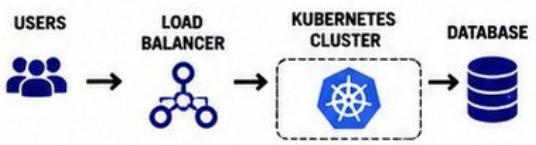
19. REAL PRODUCTION SRE INCIDENT LAB

A REALISTIC PRODUCTION SCENARIO CONNECTING THE HANDBOOK CONCEPTS.

INCIDENTS WILL HAPPEN.
HOW WE RESPOND DETERMINES OUR RELIABILITY.

SCENARIO

An e-commerce platform experiences a major traffic surge during a flash sale. Users begin to see errors and timeouts. The SRE team must respond quickly, restore service, and learn from the incident.



1 TRAFFIC SUDDENLY INCREASES

- Marketing campaign goes live.
- Traffic increases by 300%.
- Systems start receiving more requests than usual.

2 APPLICATION LATENCY RISES

- Response time jumps from 120ms to 900ms.
- Users experience slowness across the platform.

3 ERROR RATE BEGINS CLIMBING

- Error rate rises from 0.1% to 5%.
- 5xx errors increase rapidly.

4 KUBERNETES PODS BECOME SATURATED

- CPU usage hits 95%+.
- Memory usage is high.
- Pod throttling and OOM kills start occurring.

5 ALERTS TRIGGER

- Prometheus alerts fire.
- PagerDuty pages the on-call engineer.
- Multiple alerts detected.

6 ERROR BUDGET BURNS RAPIDLY

- SLO: 99.9% availability.
- Error budget depletion accelerates.
- Risk of SLO breach.

7 SRE INCIDENT RESPONSE

- On-call engineer acknowledges the alert.
- Creates incident in tracking system.
- Assembles war room.

8 METRICS, LOGS & TRACES INVESTIGATION

- Check dashboards (CPU, latency, errors).
- Review logs for exceptions.
- Use traces to find slow down points.

9 IMMEDIATE MITIGATION

- Scale deployments up.
- Enable horizontal pod autoscaling.
- Increase resource limits.

10 ROOT CAUSE ANALYSIS

- Database connection pool exhausted.
- Inefficient SQL under heavy load.
- Missing index caused slow queries.

11 RECOVERY

- Fix query and add index.
- Restart affected pods.
- System stabilizes.
- Metrics return to normal.

12 POSTMORTEM

- Conduct blameless postmortem.
- Document timeline, root cause, and impact.
- Share learnings.

13 PREVENTING RECURRENCE

- Add database query performance tests.
- Set better alerts and SLOs.
- Improve autoscaling policies.

14 IMPROVE & AUTOMATE

- Automate scaling rules.
- Add runbook automation.
- Implement load testing in CI/CD pipeline.

15 CONTINUOUS LEARNING

- Update runbooks.
- Train the team.
- Refine monitoring and error budgets.
- Build a stronger, more resilient system.

INCIDENT TIMELINE (EXAMPLE)

10:00	Traffic surge starts
10:02	Latency increases
10:03	Error rate climbs
10:04	Alerts fire
10:05	On-call engineer paged
10:07	Investigation begins
10:10	Mitigation applied
10:18	Root cause identified
10:28	Fix deployed
10:35	System recovered
11:00	Postmortem starts

IMPACT SUMMARY

Duration: 35 minutes
Users Affected: 28%
Requests Failed: 1.2M
Revenue Impact: $18,500
Error Budget Burned: 82%

KEY LESSON

Reliability is built before incidents happen. Monitoring, automation, and learning turn failures into stronger systems.

INCIDENTS ARE INEVITABLE. OUTAGES ARE OPTIONAL.
Respond fast. Communicate clearly. Learn deeply. Improve continuously.

VERIQTA

ENGINEERING EXCELLENCE

20. ENTERPRISE SRE FINAL REVIEW

BUILD RELIABILITY. OPERATE EXCELLENCE. DELIVER VALUE CONTINUOUSLY.

RELIABILITY IS BUILT. EXCELLENCE IS SUSTAINED. IMPACT IS DELIVERED.

1. END-TO-END SRE OPERATING MODEL

DEFINE → **MEASURE** → **ANALYZE** → **IMPROVE** → **AUTOMATE** → **OPERATE**

- **DEFINE**: Define reliability goals and user experience.
- **MEASURE**: Instrument and measure with SLIs and metrics.
- **ANALYZE**: Analyze signals and understand risks.
- **IMPROVE**: Prioritize work and reduce the risk.
- **AUTOMATE**: Automate operations and reduce toil.
- **OPERATE**: Run reliable services and respond fast.

SRE CORE PILLARS

- Service Level Objectives
- Monitoring & Observability
- Incident Management
- Toil Reduction
- Capacity & Performance
- Change & Release Reliability

2. PRODUCTION RELIABILITY CHECKLIST

- [x] SLIs are defined and measured
- [x] SLOs have clear targets
- [x] Error budgets are in use
- [x] Alerts are actionable
- [x] Dashboards are effective
- [x] Runbooks are up to date
- [x] On-call rotations are healthy
- [x] Postmortems drive action
- [x] Capacity is planned
- [x] Backups are tested
- [x] DR plan is validated

RELIABILITY IS EVERYONE'S RESPONSIBILITY.

3. SLI, SLO, SLA REVIEW

SLI EXAMPLES

Availability
Latency (p95, p99)
Error rate
Traffic success rate
Throughput

SLO EXAMPLES

Availability: 99.9%
Latency (p95): < 300ms
Error rate: < 0.1%
Success rate: > 99.5%

SLA EXAMPLES

Uptime: 99.9%
Response time: < 1 hour
Major incident response: 15 min

MEASURE WHAT MATTERS. COMMIT WHAT YOU CAN DELIVER.

4. ERROR BUDGET REVIEW

28%

BUDGET REMAINING

- [x] Budget remaining: 28%
- [x] Consumed this period: 72%
- [x] Burn rate (1h): 2.3x
- [x] Burn rate (6h): 1.1x
- [x] Budget policy enforced
- [x] Feature work paused? YES
- [x] Reliability work prioritized

PROTECT THE BUDGET. PROTECT THE USERS.

5. OBSERVABILITY CHECKLIST

- [x] Golden signals are monitored
- [x] SLIs are visible and trusted
- [x] Dashboards show key health
- [x] Alerts are meaningful
- [x] Logs are centralized
- [x] Traces are collected
- [x] Business metrics are tracked
- [x] Synthetic checks are in place
- [x] Dependencies are monitored
- [x] Data retention is adequate
- [x] Observability is cost-effective

YOU CAN'T IMPROVE WHAT YOU CAN'T SEE.

6. INCIDENT READINESS CHECKLIST

- [x] On-call rotations are documented
- [x] Escalation policies are clear
- [x] Runbooks are available and tested
- [x] Communication templates ready
- [x] War room process defined
- [x] Status page automation enabled
- [x] Paging is tested regularly
- [x] Postmortem template ready
- [x] Blameless culture is reinforced
- [x] Stakeholder contacts are updated

PREPARE BEFORE THE ALARM. BE READY TO RESPOND.

7. ON-CALL READINESS

- Healthy rotation and fair load
- Sufficient overlap and handoff
- On-call training and shadowing
- Tools are accessible 24/7
- Fatigue is monitored
- Time off after major incidents
- Psychological safety assured

SUSTAINABLE ON-CALL. HEALTHY ENGINEERS.

8. RELIABILITY AUTOMATION

- Runbook automation implemented
- Auto-scaling is in place
- Self-healing is enabled
- Auto-remediation for common issues
- CI/CD with safety gates
- Automated rollback on failure
- ChatOps and automation tools

AUTOMATE TO ELIMINATE TOIL. FOCUS ON ENGINEERING.

9. COMMON SRE MISTAKES

- [x] Alerting on symptoms, not causes
- [x] No SLOs or error budgets
- [x] Ignoring to reduce toil
- [x] Too many noisy alerts
- [x] No postmortems or no action
- [x] Skipping capacity planning
- [x] Manual, repetitive processes
- [x] No disaster recovery testing
- [x] Hero culture and burnout
- [x] Not measuring user experience

AVOID MISTAKES. BUILD MATURITY.

10. SENIOR SRE PRINCIPLES

- User-centric reliability
- Data-driven decisions
- Reduce risk continuously
- Automate ruthlessly
- Learn from every failure
- Design for failure
- Think long-term
- Enable the team
- Communicate clearly
- Drive business impact

LEAD WITH IMPACT. THINK LIKE AN OWNER.

11. SRE MATURITY ROADMAP

1. **INITIAL** — Ad-hoc processes. Manual operations. Firefighting mode.
2. **DEVELOPING** — Basic monitoring. Some runbooks. On-call established.
3. **DEFINED** — SLIs, SLOs defined. Postmortems used. Automation begins.
4. **ADVANCED** — Strong reliability culture. Automation at scale. Proactive operations.
5. **OPTIMIZED** — Continuous improvement. Predictive reliability. Business outcomes.

MATURITY IS A JOURNEY. IMPROVEMENT NEVER STOPS.

12. FINAL PRODUCTION RECOMMENDATIONS

- [x] Define clear reliability goals aligned to business outcomes.
- [x] Measure everything. Optimize what matters.
- [x] Protect your error budget and SLOs.
- [x] Invest in observability and meaningful alerts.
- [x] Reduce toil with automation and better tooling.
- [x] Prepare for incidents. Practice regularly.
- [x] Learn from failures and drive action.
- [x] Design for scale, resilience, and change.
- [x] Build a culture of reliability and ownership.
- [x] Keep improving. Deliver exceptional user experience.

RELIABILITY IS A COMPETITIVE ADVANTAGE.

RELIABILITY IS HOW WE EARN TRUST. EXCELLENCE IS HOW WE KEEP IT.